

```
#
=====
=====
# IMPORTS
#
=====
=====

from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS

Afficher plus ▾
```

J'ai bien reçu ton code Flask. Que souhaites-tu que je fasse avec ?

Je peux par exemple :

- 🔍 **faire une revue de code** (bugs, sécurité, bonnes pratiques Flask/SQLAlchemy)
- 🐛 **corriger un problème précis** (erreur, route qui ne marche pas, base SQLite, CORS, React...)
- 🚀 **améliorer l'API** (pagination, recherche, filtres, gestion d'erreurs, migrations)
- 🧩 **expliquer le fonctionnement du code partie par partie**
- 🔄 **adapter l'API pour un frontend React**
- 📝 **ajouter des tests**
- 📦 **préparer une structure de projet professionnelle Flask**

À première vue, ton API est déjà bien structurée : CRUD complet `/trips` , modèle SQLAlchemy moderne avec `Mapped` , validation des entrées, gestion d'images Unsplash et CORS. Quelques points pourraient toutefois être améliorés (par exemple la validation `PUT` qui oblige tous les champs, la gestion des exceptions SQL, la sécurité de `debug=True` , et la validation plus stricte de l'URL Unsplash).

📄 👍 💬 ↗️ ↺ ...

🧩 expliquer le fonctionnement du code partie par partie

Je vais t'expliquer ton API Flask **partie par partie**, dans l'ordre où le code est écrit, avec le rôle de chaque bloc.

1) Les imports

```
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS
from urllib.parse import urlparse
```

Python



Exécuter

Ces lignes importent les outils nécessaires.

Flask

Python



Exécuter

Permet de créer ton application web.

Exemple :

Python



Exécuter

crée ton serveur API.

jsonify

Python



Exécuter

Transforme un dictionnaire Python en réponse JSON.

Exemple :

</> Python



▶ Exécuter

renvoie :

{

</> JSON



request

</> Python



▶ Exécuter

Permet de récupérer les données envoyées par le client.

Par exemple, React peut envoyer :

```
{  
  "title": "Paris",
```

</> JSON



Flask récupère ces données avec :

</> Python



▶ Exécuter

SQLAlchemy

</> Python



▶ Exécuter

C'est l'ORM utilisé pour communiquer avec la base de données.

ORM signifie :

Object Relational Mapping

Au lieu d'écrire :

```
</> SQL
```



tu écris :

```
</> Python
```



▶ Exécuter

CORS

```
</> Python
```



▶ Exécuter

Autorise ton frontend React à communiquer avec ton backend Flask.

Sans CORS :

```
React : localhost:3000
  |
  X
  |
Flask : localhost:5000
```



Le navigateur bloque la requête.

Avec :

```
</> Python
```



▶ Exécuter

la communication est autorisée.

urlparse

</> Python



▶ Exécuter

Permet d'analyser une URL.

Exemple :

```
urlparse(
```

</> Python



▶ Exécuter

donne :

```
scheme = https
netloc = images.unsplash.com
path = /photo-123
```



Tu l'utilises pour vérifier les images Unsplash.

2) Création de l'application Flask

</> Python



▶ Exécuter

Crée ton application.

`__name__` indique à Flask où trouver les fichiers de ton projet.

3) Activation de CORS

</> Python



▶ Exécuter

Autorise les requêtes venant du frontend.

Exemple :

React :

```
GET http://localhost:5000/trips
```



Flask accepte cette requête.

4) Configuration de la base de données

```
</> Python
```



▶ Exécuter

Indique à Flask quelle base utiliser.

Ici :

```
SQLite
```



Le fichier créé sera :

```
instance/  
travel.db
```



```
</> Python
```



▶ Exécuter

Désactive une fonctionnalité inutile de SQLAlchemy.

Cela économise de la mémoire.

5) Création de la classe Base SQLAlchemy

```
</> Python
```



▶ Exécuter

Cette classe sert de modèle parent pour tes tables.

Toutes tes classes SQLAlchemy vont hériter de cette base.

Exemple :

```
Base
|
└─ Trip
```



6) Initialisation SQLAlchemy

```
db = SQLAlchemy(model_class=Base)
```

</> Python



▶ Exécuter

Crée l'objet qui permet de manipuler la base.

Grâce à db , tu peux :

Créer :

</> Python



▶ Exécuter

Lire :

</> Python



▶ Exécuter

Supprimer :

</> Python



▶ Exécuter

Sauvegarder :

</> Python



▶ Exécuter

7) Le modèle Trip

Cette partie représente ta table SQL.

Python



Exécuter

Créera une table :

trips



dans SQLite.

Nom de la table

Python



Exécuter

Nom exact dans la base :

trips



8) Les colonnes de la table

Identifiant



ChatGPT



Mettre à niveau



Exécuter



Correspond à :

SQL :

id INTEGER PRIMARY KEY



Chaque voyage possède un numéro unique.

Exemple :


```
1 Paris
2 Tokyo
3 Rome
```



Informations générales

</> Python



▶ Exécuter

```
title
destination
country
description
```

Créent :

Colonne	Type	
title	texte	
destination	texte	
country	texte	
description	texte	

Exemple :

```
title = "Voyage à Tokyo"
country = "JP"
```



Informations voyage

</> Python



▶ Exécuter

```
price
duration
travel_type
```

Exemple :

</> JSON



```
{  
  "price":1200,  
  "duration":7,  
  "travel_type":"city"  
}
```

Image

</> Python



▶ Exécuter

```
image_filename
```

Tu ne stockes pas toute l'URL.

Tu stockes seulement :

```
/photo-12345
```



Puis tu reconstruis :

```
https://images.unsplash.com/photo-12345
```



C'est plus propre.

Disponibilité

</> Python



▶ Exécuter

```
is_available
```

Indique si le voyage est disponible.

Exemple :

</> JSON



```
"is_available": true
```

9) La méthode to_dict()

</> Python



▶ Exécuter

```
def to_dict(self):
```

Transforme un objet Python en JSON.

Avant :

</> Python



▶ Exécuter

```
Trip(  
title="Paris"  
)
```

Après :

</> JSON



```
{  
  "title": "Paris"  
}
```

Elle est utilisée avant chaque réponse API.

10) Création automatique des tables

</> Python



▶ Exécuter

```
with app.app_context():  
    db.create_all()
```

Demande à SQLAlchemy :

"Crée toutes les tables qui n'existent pas."

Résultat :

travel.db



```
trips  
|  
|-- id  
|-- title  
|-- price  
|-- duration
```

11) Fonction `validate_body()`

Cette fonction vérifie les données envoyées par React.

Exemple :

Le frontend envoie :

```
</> JSON

{
  "title": "Paris",
  "price": 500
}
```



La fonction vérifie :

- Est-ce du JSON ?
- Tous les champs existent-ils ?
- Les types sont-ils corrects ?
- Les valeurs sont-elles acceptables ?

Vérification des champs obligatoires

```
</> Python
```



▶ Exécuter

```
mandatory_fields
```

Liste :

```
title
destination
country
description
price
duration
travel_type
image
```



Si un champ manque :

```
</> Python
```



▶ Exécuter

```
return "Le champ est requis"
```

12) Validation du pays

</> Python



▶ Exécuter

```
body["country"].strip().upper()
```

Transforme :

fr



en :

FR



Puis vérifie :

2 caractères



13) Validation prix/durée

Prix :

</> Python



▶ Exécuter

```
if body["price"] <= 0
```

Empêche :

price = -50



Durée :

</> Python



▶ Exécuter

```
duration <=0
```

Empêche :

duration = 0



14) Validation du type de voyage

Types acceptés :

</> Python



▶ Exécuter

```
[  
  "city",  
  "nature",  
  "beach",  
  "adventure"  
]
```

Donc :

Accepté :

</> JSON



```
{  
  "travel_type": "beach"  
}
```

Refusé :

</> JSON



```
{  
  "travel_type": "space"  
}
```

15) Validation image Unsplash

</> Python



▶ Exécuter

```
parsed = urlparse(body["image"])
```

Analyse l'URL.

Puis :

</> Python



▶ Exécuter

```
if "images.unsplash.com" not in parsed.netloc
```

refuse :

`https://google.com/image.jpg`



Accepte :

`https://images.unsplash.com/photo-123`



16) Route d'accueil

Python



▶ Exécuter

```
@app.route("/")
```

Quand tu visites :

`localhost:5000/`



Tu obtiens :

Bienvenue sur mon API de voyages !



C'est juste un test.

17) GET /trips

Python



▶ Exécuter

```
@app.route("/trips")
```

Retourne tous les voyages.

Le client demande :

`GET /trips`



Flask fait :

Python



▶ Exécuter

```
db.select(Trip)
```

équivalent SQL :

</> SQL



```
SELECT * FROM trips;
```

Puis renvoie :

</> JSON



```
[  
{  
  "id":1,  
  "title":"Paris"  
}]
```

18) GET /trips/<id>

Exemple :

GET /trips/3



Cherche :

</> Python



Exécuter

```
db.session.get(Trip,3)
```

Si trouvé :

</> JSON



```
{  
  "id":3,  
  "title":"Rome"  
}
```

Sinon :

</> JSON



```
{  
  "error":"Trip not found"  
}
```

avec code HTTP :

404



19) POST /trips

Créer un voyage.

Le frontend envoie :

POST /trips



avec :

```
</> JSON
{
  "title": "Tokyo",
  "price": 1500
}
```



Étapes :

1. récupérer JSON

</> Python



Exécuter

```
body=request.get_json()
```

2. vérifier les données

</> Python



Exécuter

```
validate_body()
```

3. créer un objet Trip

</> Python



Exécuter

```
trip=Trip(...)
```

4. ajouter en base

</> Python



Exécuter

```
db.session.add(trip)
```

5. sauvegarder

</> Python



▶ Exécuter

```
db.session.commit()
```

6. retourner le voyage créé

Code :

201 Created



20) PUT /trips/<id>

Modifier un voyage.

Exemple :

PUT /trips/2



Cherche le voyage :

</> Python



▶ Exécuter

```
db.session.get()
```

Puis remplace ses valeurs :

</> Python



▶ Exécuter

```
trip.title = ...
```

Enfin :

</> Python



▶ Exécuter

```
db.session.commit()
```

21) DELETE /trips/<id>

Supprime un voyage.

Exemple :

DELETE /trips/5



Recherche :

</> Python



▶ Exécuter

```
trip=db.session.get()
```

Suppression :

</> Python



▶ Exécuter

```
db.session.delete(trip)
```

Sauvegarde :

</> Python



▶ Exécuter

```
db.session.commit()
```

Retour :

204 No Content



22) Démarrage du serveur

</> Python



▶ Exécuter

```
if __name__ == "__main__":
```

Signifie :

"Exécute le serveur seulement si ce fichier est lancé directement."

</> Python



▶ Exécuter

```
app.run(  
host="127.0.0.1",  
port=5000,  
debug=True  
)
```

Lance Flask :

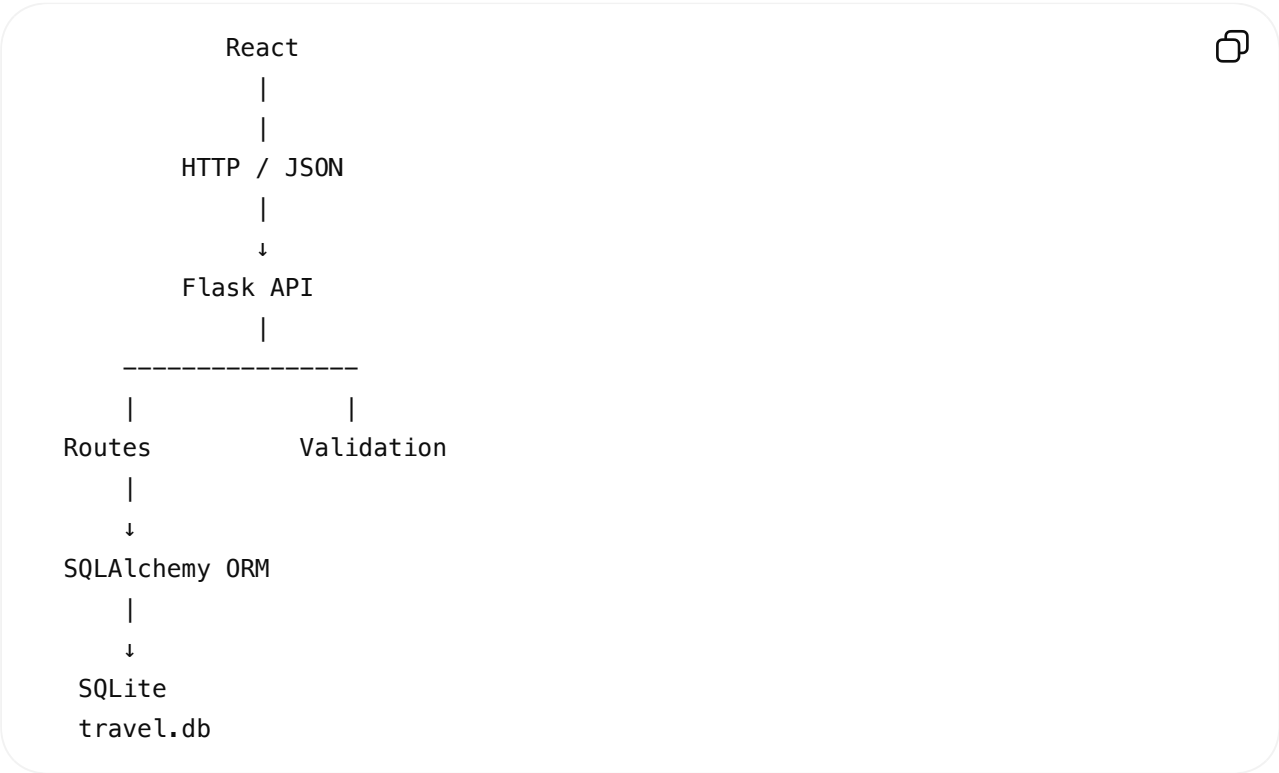
Adresse :

http://127.0.0.1:5000

Mode debug :

- recharge automatique
- messages d'erreur détaillés

Vue globale de ton architecture



Ton API réalise donc un **CRUD complet** :

Action	Méthode	Route
Lire tous les voyages	GET	/trips
Lire un voyage	GET	/trips/<id>
Créer	POST	/trips
Modifier	PUT	/trips/<id>
Supprimer	DELETE	/trips/<id>

En résumé : ton code est une **API REST Flask complète pour gérer une base de voyages**, prête à être consommée par un frontend React.

